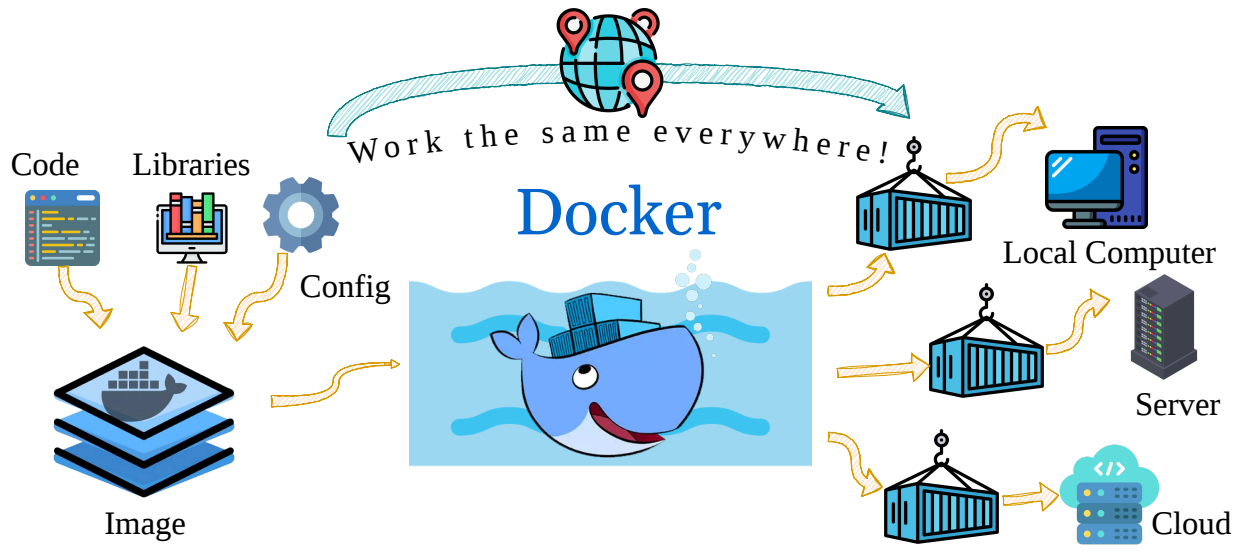
Packaging AI Inference with Docker (Nâng cao)

Tạ Hữu Anh Dương **Dương Nguyễn Thuận**

Proofreading: Đinh Quang Vinh

I. Giới thiệu

Trong quá trình tìm hiểu các dự án mã nguồn mở trên GitHub, người học thường gặp những công cụ hữu ích và muốn chạy thử. Tuy nhiên, việc cài đặt một dự án mới thường đòi hỏi chuẩn bị nhiều thành phần đi kèm như runtime, thư viện và các dependency với phiên bản cụ thể. Những thành phần này không chỉ tốn thời gian thiết lập mà còn dễ xung đột với môi trường sẵn có, khiến máy tính cá nhân dần tích tụ nhiều cấu hình không còn sử dụng. Bên cạnh đó, hướng dẫn cài đặt thường được tách riêng cho từng hệ điều hành như Windows, macOS và Linux, và kết quả chạy trên các máy khác nhau đôi khi không đồng nhất, dẫn đến tình huống quen thuộc "chạy được trên máy cá nhân nhưng không chạy được trên máy khác". Đáng chú ý, ngày càng nhiều dự án bổ sung thêm một lựa chọn cài đặt bên cạnh các hệ điều hành trên, đó là **Docker**.



Hình 1: Tổng quan về Docker: mã nguồn, thư viện và cấu hình được đóng gói thành image, sau đó chạy dưới dạng container trên các môi trường khác nhau như máy tính cá nhân, server và cloud.

Vậy Docker làm điều đó như thế nào? Trước tiên, hãy cùng xem một ứng dụng phải trải qua những bước nào để được triển khai. Câu trả lời nằm ở cách Docker tiếp cận vấn đề. Thay vì yêu cầu người dùng tự cấu hình môi trường trên máy của mình, Docker cho phép đóng gói mã nguồn, thư viện và cấu hình của ứng dụng thành một image thống nhất. Image này sau đó được khởi chạy dưới dạng container, một môi trường cô lập với hệ thống bên ngoài, và cho kết quả nhất quán trên mọi máy có cài đặt Docker, từ máy tính cá nhân, server cho đến hạ tầng cloud (Hình 1).

Bảng Giải Thích Thuật Ngữ

OS Kernel	Phần lõi của hệ điều hành, quản lý tiến trình, bộ nhớ, phần cứng và cung cấp system call cho phần mềm.
Namespaces	Tính năng của Linux kernel giới hạn tài nguyên mà tiến trình nhìn thấy (PID, network, mount...), giúp cô lập container dù vẫn dùng chung kernel.
Cgroups (Control Groups)	Tính năng của Linux kernel giới hạn và giám sát tài nguyên (CPU, RAM, I/O) của một nhóm tiến trình, ví dụ một container.
User space	Môi trường chạy ứng dụng ở chế độ không đặc quyền, chỉ truy cập phần cứng gián tiếp qua system call.
SIGTERM / SIGKILL	SIGTERM yêu cầu tiến trình dừng và cho phép dọn dẹp trước khi thoát; SIGKILL buộc dừng ngay, không thể bị chặn.
OCI (Open Container Initiative)	Dự án thuộc Linux Foundation, đặt tiêu chuẩn mở cho định dạng image, runtime và cách phân phối container.
NAT (Network Address Translation)	Kỹ thuật dịch địa chỉ IP và cổng, cho phép container trong mạng ảo dùng IP của host để ra ngoài và nhận kết nối qua cổng được publish.
VXLAN	Giao thức đóng gói frame Layer 2 vào gói UDP để truyền qua mạng Layer 3, dùng tạo overlay network giữa nhiều host.
ONNX Runtime	Inference engine mã nguồn mở, chạy và tăng tốc mô hình định dạng ONNX trên nhiều phần cứng (CPU, GPU).

Mục lục

I. Giới thiệu **1**

II. Từ Local Deployment đến Container Deployment **4**

III. Kiến trúc và cơ chế hoạt động của Docker **5**

III.1. Mô hình kiến trúc tổng quan 5

III.2. Containerd and runc 6

IV. Các khái niệm cốt lõi trong Docker **7**

IV.1. Dockerfile & Docker Image 7

IV.1.1. Dockerfile 7

IV.1.2. Docker Image 10

IV.2. Docker Container 11

IV.3. Docker Registry 13

IV.4. Docker Volume & Bind Mount 14

IV.5. Docker Network 15

IV.5.1. Các loại network trong Docker 16

IV.6. Docker Compose 18

V. Thực hành **21**

V.1. Kiến trúc tổng quan 22

V.1.1. Vai trò từng thành phần 22

V.1.2. Dockerfile của backend 23

V.1.3. Dockerfile của frontend 24

V.1.4. File docker-compose.yml 24

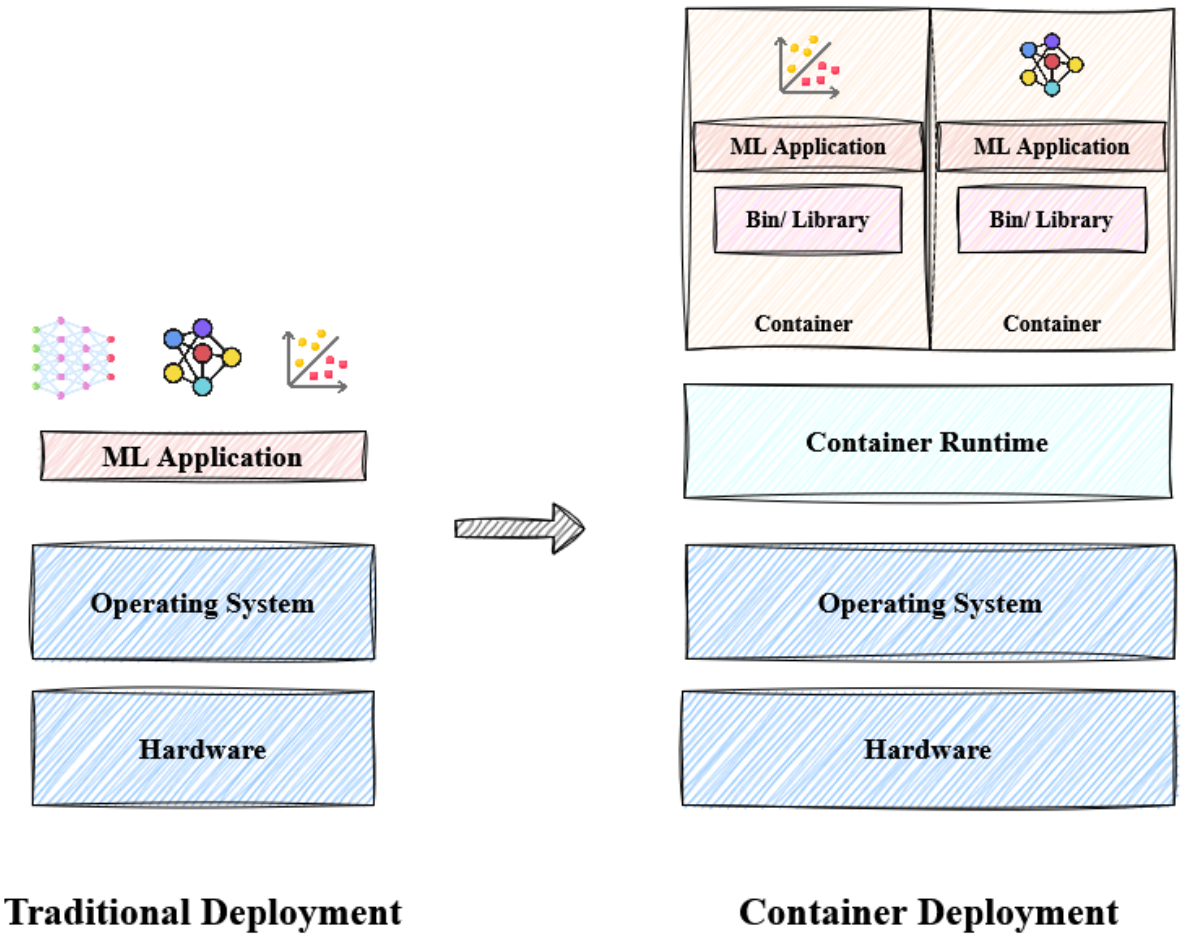
V.1.5. Cấu trúc thư mục 26

VI. Triển khai **26**

VII. Tài liệu tham khảo **28**

II. Từ Local Deployment đến Container Deployment

Giả sử một nhóm phát triển cần triển khai hai mô hình ML lên cùng một máy chủ: mô hình thứ nhất được huấn luyện với PyTorch 1.13 và CUDA 11.7, trong khi mô hình thứ hai yêu cầu PyTorch 2.3 và CUDA 12.1. Cách hai phương thức triển khai xử lý tình huống này phản ánh rõ sự khác biệt giữa chúng (Hình 2).



Hình 2: So sánh kiến trúc Local Deployment (trái) và Container Deployment (phải).

Với **Local Deployment**, cả hai mô hình được cài đặt trực tiếp lên hệ điều hành của máy chủ và dùng chung một môi trường. Do hệ thống chỉ có thể giữ một phiên bản thư viện tại một thời điểm, việc cài đặt mô hình thứ hai sẽ ghi đè lên các thư viện mà mô hình thứ nhất đang sử dụng. Nhóm phát triển buộc phải chọn giữa việc tìm cách cho hai mô hình cùng tồn tại một cách thủ công, hoặc thuê thêm máy chủ riêng cho từng mô hình. Ngay cả khi triển khai thành công, môi trường trên máy chủ cũng khó có thể tái tạo chính xác ở nơi khác, bởi nó là kết quả của hàng loạt lần cài đặt và chỉnh sửa tích lũy theo thời gian.

Với **Container Deployment**, mỗi mô hình được đóng gói cùng phiên bản thư viện riêng của nó vào một container. Hai container chạy song song trên cùng máy chủ, dùng chung kernel của hệ điều hành nhưng hoàn toàn tách biệt về thư viện, file hệ thống và tiến trình, nên không xảy ra xung đột. Môi trường lúc này không còn phụ thuộc vào trạng thái của máy chủ mà được mô

tả và đóng gói cùng ứng dụng, do đó có thể mang sang bất kỳ máy nào khác và cho kết quả như nhau.

III. Kiến trúc và cơ chế hoạt động của Docker

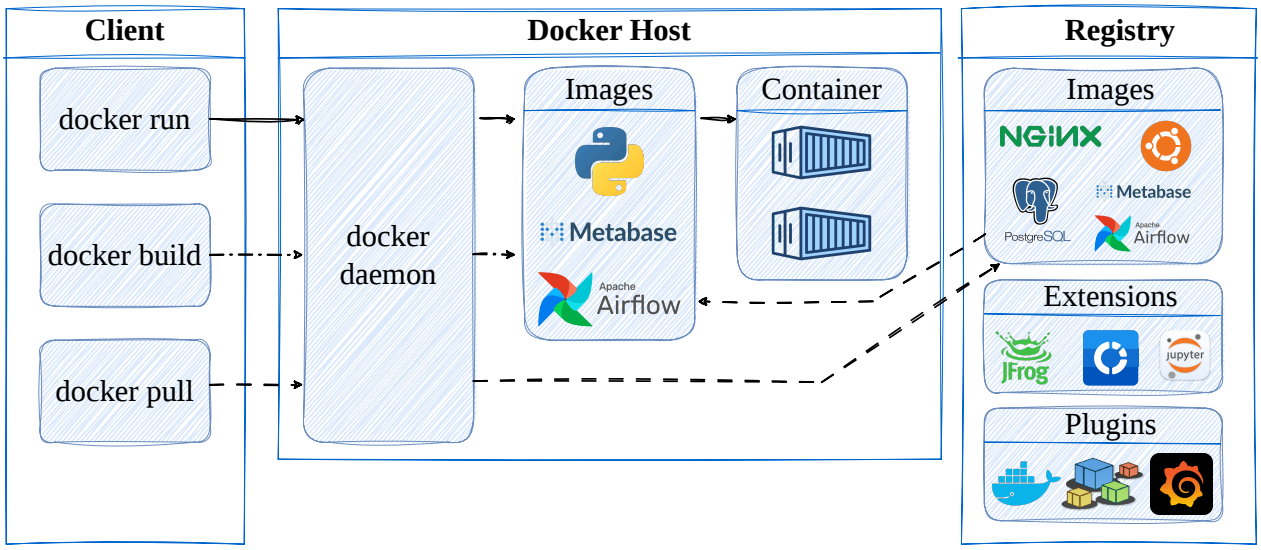
III.1. Mô hình kiến trúc tổng quan

Docker vận hành dựa trên kiến trúc client-server. Trong mô hình này, Docker Client không trực tiếp quản lý hay thực thi các container; thay vào đó, nó giao tiếp với Docker Daemon (thành phần đảm nhận các tác vụ chính như xây dựng, chạy và phân phối các container) trên máy chủ Docker Host thông qua REST API, socket UNIX cục bộ hoặc các giao thức mạng tiêu chuẩn.

Lưu ý

Docker client và docker daemon có thể chạy trên cùng một hệ thống, hoặc chúng ta có thể kết nối Docker client với một Docker daemon ở xa.

Một Docker Client khác là Docker Compose; đây là một công cụ cho phép người dùng làm việc với các ứng dụng bao gồm nhiều container.



Hình 3: Kiến trúc phân tầng của Docker và luồng tương tác giữa Docker Client, Docker Host và Docker Registry.

Như mô tả trong Hình 3, hệ thống Docker bao gồm ba khối chức năng chính:

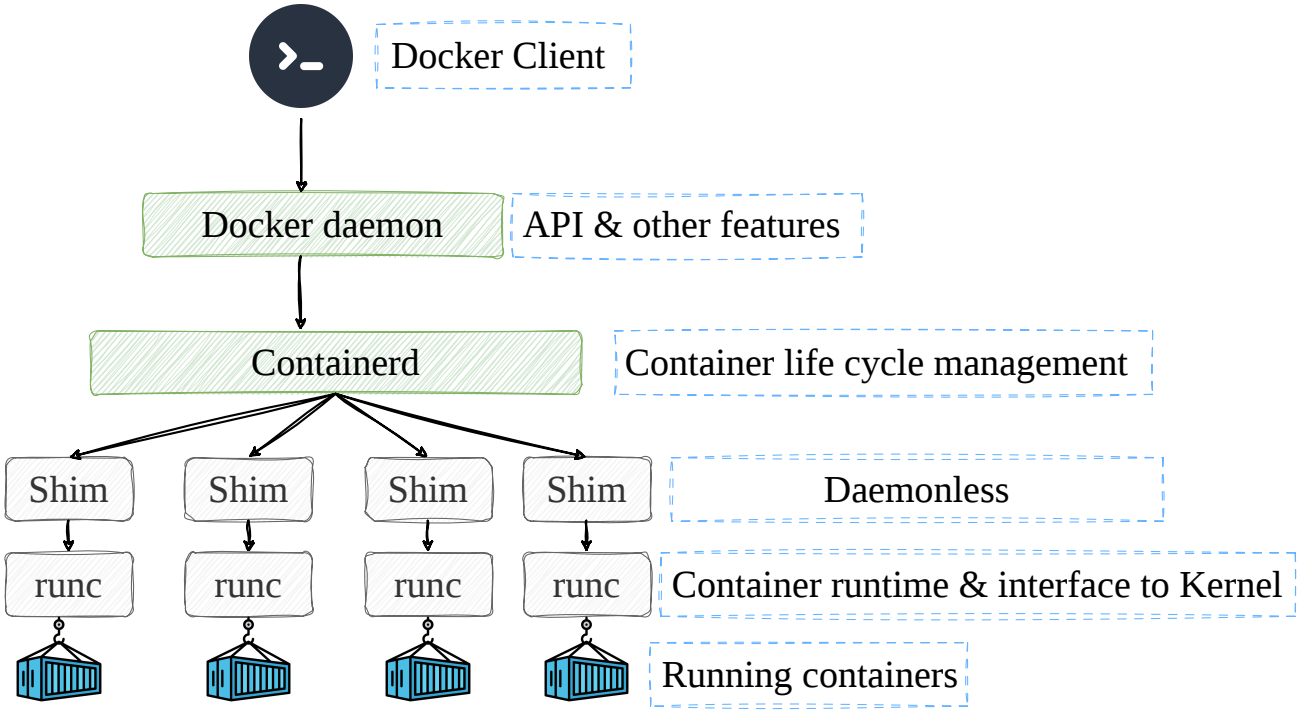
1. **Docker Client:** Cung cấp giao diện dòng lệnh (`CLI`) để người dùng tương tác với hệ thống. Khi người dùng thực thi các câu lệnh như `docker build`, `docker pull` hoặc `docker run`, Client sẽ mã hóa các chỉ thị này thành các yêu cầu HTTP/REST API tương ứng gửi tới Docker Daemon.

2. **Docker Host:** Môi trường thực thi chính, nơi tiếp nhận, phân tích cú pháp và xử lý các chỉ thị từ Client. Docker Host đảm nhiệm việc quản lý toàn bộ vòng đời của các tài nguyên hệ thống (bao gồm images, container, phân vùng lưu trữ và mạng ảo).

3. **Docker Registry:** Hệ thống máy chủ lưu trữ và phân phối các Docker Image. Registry có thể hoạt động ở chế độ công cộng (như Docker Hub) hoặc triển khai nội bộ. Docker Hub đồng thời cung cấp các tiện ích mở rộng và các plugin nhằm mở rộng khả năng quản trị hạ tầng container.

III.2. Containerd and runc

Trong khi Docker Daemon xử lý các tác vụ cấp cao, nó ủy thác các hoạt động cấp thấp liên quan đến container cho các công cụ chuyên dụng như containerd, shim, và runc.



Hình 4: Kiến trúc phân lớp chi tiết của Docker Engine.

- **Containerd:** là một tiến trình daemon giúp quản lý toàn bộ vòng đời của container (tải image, tạo container, quản lý lưu trữ). Docker sử dụng containerd ở tầng bên dưới, nhưng người dùng có thể tương tác trực tiếp với containerd thông qua giao diện dòng lệnh (CLI) của nó là 'ctr'.

- **runc (Low-Level Container Runtime):** Là runtime cấp thấp, thực thi nhiệm vụ cuối cùng. Nó trực tiếp giao tiếp với kernel của hệ điều hành để thiết lập namespaces, cgroups và khởi tạo tiến trình container. Sau khi tạo xong container, runc sẽ tự động thoát ra và chuyển giao việc quản lý lại cho Shim.

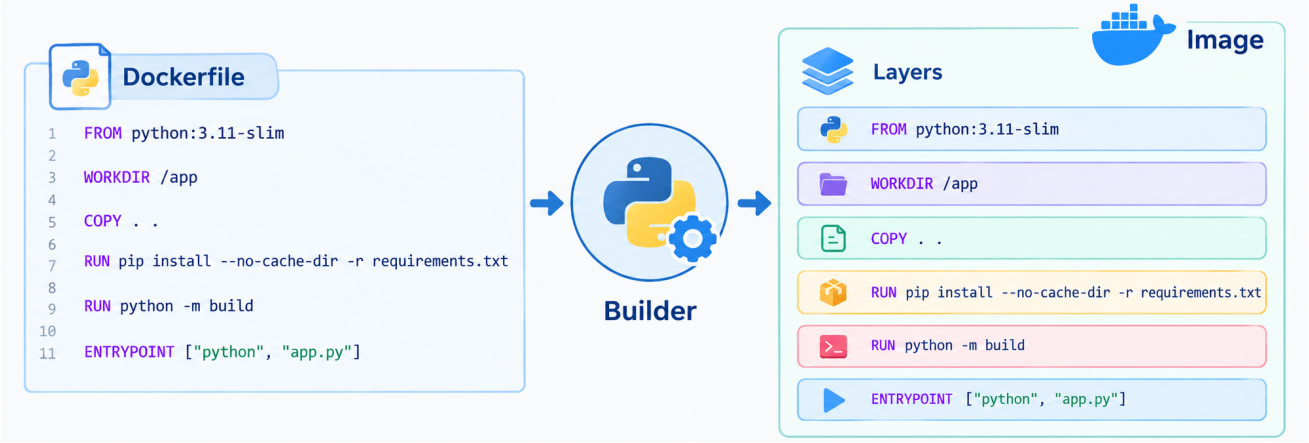
- **Containerd Shims:** Đóng vai trò là tiến trình đảm bảo các container không bị mất đi khi tiến trình runc kết thúc. Shim giúp tách biệt tiến trình của container khỏi Docker Engine,

cho phép các container tiếp tục chạy độc lập ngay cả khi containerd hoặc dockerd khởi động lại hoặc gặp sự cố.

IV. Các khái niệm cốt lõi trong Docker

IV.1. Dockerfile & Docker Image

Dockerfile và Docker Image là hai khái niệm thường được nhắc đến cùng nhau. Dockerfile là tệp văn bản mô tả từng bước dựng môi trường chạy ứng dụng, còn image là sản phẩm thu được sau khi Docker thực thi các bước đó. Quá trình chuyển đổi được thực hiện bằng lệnh `docker build`: thành phần builder đọc Dockerfile, thực thi lần lượt từng instruction và ghi kết quả thành các layer xếp chồng lên nhau bên trong image (Hình 5). Có thể so sánh Dockerfile với mã nguồn và image với chương trình đã được biên dịch: cùng một Dockerfile, khi build trên các máy khác nhau, sẽ cho ra những môi trường giống nhau.



Hình 5: Quá trình build image. Builder đọc Dockerfile, thực thi tuần tự từng instruction và lưu kết quả thành các layer của image theo đúng thứ tự trong Dockerfile.

IV.1.1. Dockerfile

Dockerfile là một tệp văn bản thuần chứa các instruction được Docker đọc và thực thi lần lượt từ trên xuống dưới để tạo ra image. Mỗi instruction có dạng `INSTRUCTION arguments`. Tên instruction không phân biệt chữ hoa và chữ thường, nhưng theo quy ước được viết hoa để dễ tách biệt với phần tham số. Instruction đầu tiên thường là `FROM`, dùng để xác định base image mà các bước tiếp theo sẽ xây dựng lên trên.

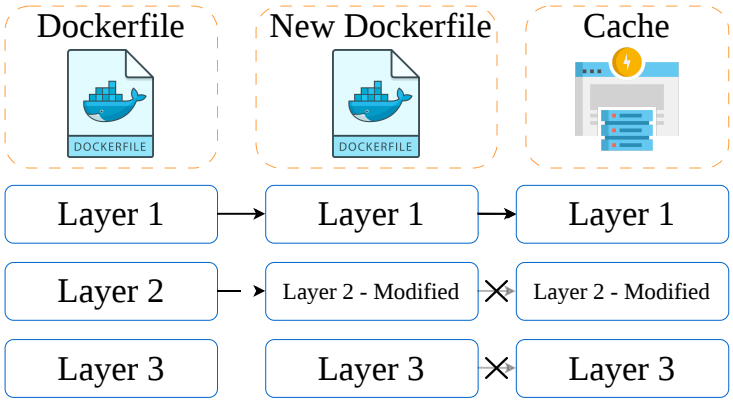
Bảng 1 tóm tắt các instruction thường gặp trong Dockerfile.

Bảng 1: Các instruction thường dùng trong Dockerfile.

Instruction	Ý nghĩa
`FROM`	Chỉ định image nền. Mọi Dockerfile hợp lệ đều bắt đầu bằng `FROM` (có thể đứng sau `ARG`).
`WORKDIR`	Thiết lập thư mục làm việc cho các chỉ thị phía sau; tự tạo thư mục nếu chưa tồn tại.
`COPY`	Sao chép tệp hoặc thư mục từ build context vào image.
`ADD`	Tương tự `COPY` nhưng có thêm khả năng tự giải nén tệp nén cục bộ và tải tệp từ URL. Chỉ nên dùng khi cần các tính năng này.
`RUN`	Thực thi lệnh trong quá trình build, ví dụ cài đặt thư viện. Kết quả được lưu thành một lớp mới.
`ENV`	Khai báo biến môi trường, có hiệu lực cả khi build và khi container chạy.
`ARG`	Khai báo biến chỉ có hiệu lực trong quá trình build, truyền giá trị qua `--build-arg`.
`EXPOSE`	Ghi chú cổng mà ứng dụng lắng nghe. Chỉ thị này mang tính tài liệu, không tự mở cổng ra ngoài.
`USER`	Chỉ định người dùng chạy các lệnh tiếp theo và tiến trình chính của container.
`VOLUME`	Khai báo điểm gắn kết dữ liệu cần được lưu trữ bên ngoài container.
`HEALTHCHECK`	Định nghĩa lệnh kiểm tra định kỳ trạng thái hoạt động của ứng dụng.
`ENTRYPOINT`	Xác định tiến trình chính của container.
`CMD`	Cung cấp lệnh mặc định, hoặc tham số mặc định cho `ENTRYPOINT`; có thể bị ghi đè khi chạy `docker run`.

Build cache. Một ứng dụng thường được build lại nhiều lần trong quá trình phát triển, và phần lớn các lần build chỉ khác nhau ở một vài dòng mã nguồn. Để tránh lặp lại những bước tốn thời gian, builder lưu kết quả của từng instruction vào bộ nhớ đệm. Ở lần build tiếp theo, trước khi thực thi một instruction, builder kiểm tra xem có thể dùng lại kết quả cũ hay không. Đây chính là lý do tại sao thứ tự các lệnh trong Dockerfile lại quan trọng.

Điểm quan trọng nhất của cơ chế này là khi một instruction bị vô hiệu hóa cache, mọi instruction đứng sau nó đều phải thực thi lại, dù bản thân chúng không thay đổi. Hình 6 minh họa trường hợp Layer 2 bị sửa đổi: Layer 1 vẫn được lấy từ cache, nhưng Layer 2 và Layer 3 đều phải build lại.



Hình 6: Cơ chế vô hiệu hóa build cache. Khi Layer 2 thay đổi, Layer 1 vẫn được dùng lại từ cache, còn Layer 2 và mọi layer phía sau phải build lại, kể cả Layer 3 vốn không có thay đổi.

Hệ quả trực tiếp là thứ tự các instruction ảnh hưởng lớn đến thời gian build. Cách khắc phục là sắp xếp các instruction theo mức độ thay đổi tăng dần: những phần ít thay đổi (cài đặt các gói hệ thống, thư viện) đặt ở trên, cho phép Docker lưu vào cache các layer, giúp các lần build sau có thể tái sử dụng chúng và tiết kiệm thời gian. Những phần thay đổi thường xuyên (như mã nguồn) được đặt ở dưới vì điều này đảm bảo rằng khi mã nguồn ứng dụng thay đổi, chỉ một số ít lớp cần được build lại, giúp quá trình này diễn ra nhanh hơn.

Ví dụ dưới đây minh họa một Dockerfile cơ bản cho ứng dụng Python sử dụng FastAPI.

Ví dụ tạo Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

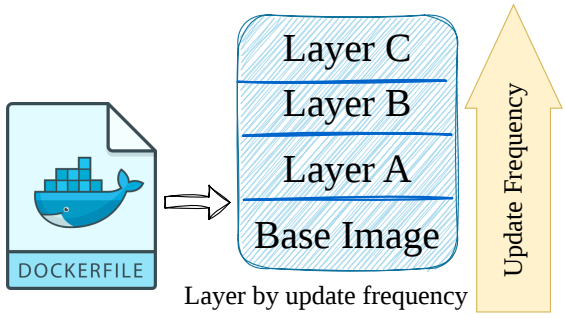
COPY . .

EXPOSE 8000

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Trong ví dụ trên, lệnh `FROM` chọn image nền `python:3.11-slim`, một phiên bản rút gọn giúp giảm dung lượng image, sau đó `WORKDIR` tạo và chuyển vào thư mục `/app` để mọi lệnh phía sau đều thực thi tại đây. Tệp `requirements.txt` được sao chép và cài đặt trước khi đưa toàn bộ mã nguồn vào bằng `COPY . .`. Lệnh `EXPOSE` chỉ mang tính ghi chú cổng mà ứng dụng lắng nghe chứ không tự mở cổng ra máy host, còn `CMD` xác định lệnh mặc định khởi chạy server với `-host 0.0.0.0` để có thể truy cập từ bên ngoài container. Sau khi hoàn thành, ta build image bằng `docker build -t my-app .` và chạy container bằng `docker run -p 8000:8000 my-app`, trong đó tham số `-p` ánh xạ cổng 8000 của máy host vào cổng 8000 của container.

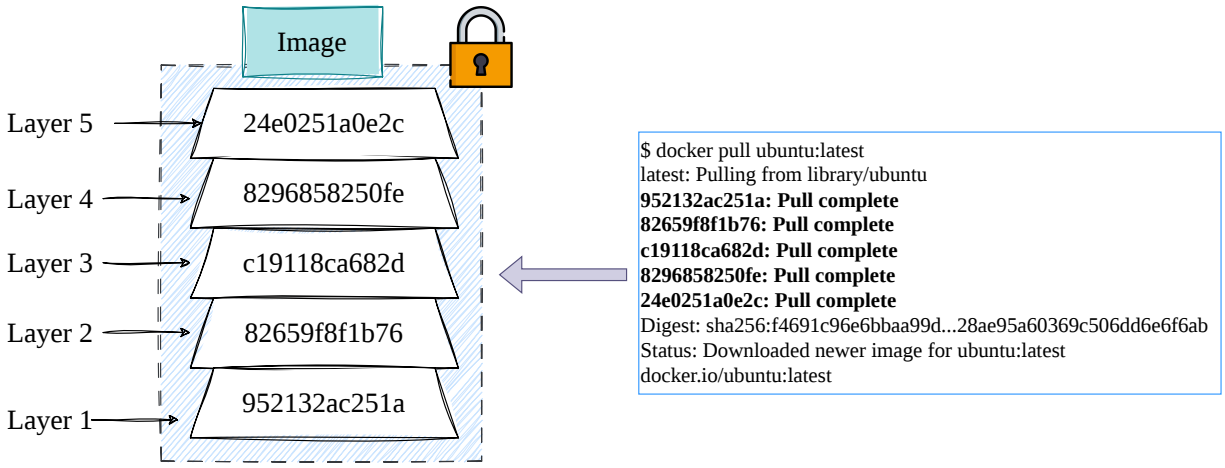
IV.1.2. Docker Image



Hình 7: Cấu trúc phân lớp của Docker image. Các layer chỉ đọc được xếp chồng theo thứ tự instruction trong Dockerfile.

Docker image là một khuôn mẫu read-only chứa mọi thứ cần thiết để chạy ứng dụng, gồm hệ thống tệp với thư viện, mã nguồn và cấu hình như biến môi trường, thư mục làm việc và lệnh khởi động mặc định. Image không phải là một tệp nguyên khối mà là tập hợp các layer xếp chồng lên nhau. Mỗi layer chứa một tập thay đổi trên hệ thống tệp so với layer bên dưới: thêm, sửa hoặc xóa tệp. Một khi đã được tạo, layer không thể bị thay đổi. Vì vậy image có tính bất biến.

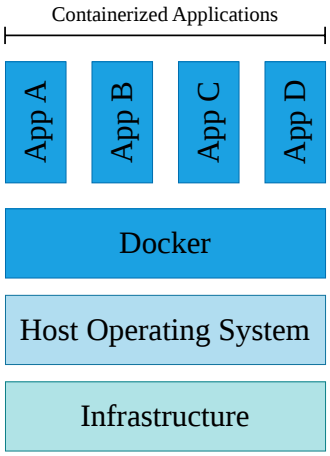
Cấu trúc phân lớp có thể quan sát trực tiếp khi tải một image về máy. Trong Hình 8, lệnh `docker pull ubuntu:latest` tải lần lượt từng layer, mỗi dòng `Pull complete` tương ứng với một layer và được hiển thị bằng 12 ký tự đầu của mã định danh và các layer này đều được lưu ở trạng thái read-only. Dòng `Digest` ở cuối là giá trị băm SHA-256 của toàn bộ image, được tính từ danh sách layer và cấu hình của image đó, cho phép Docker kiểm tra rằng image tải về đúng với image đã được công bố trên registry.



Hình 8: Cấu trúc phân lớp của image `ubuntu:latest`: mỗi layer được tải riêng, định danh bằng mã băm và lưu ở trạng thái read-only.

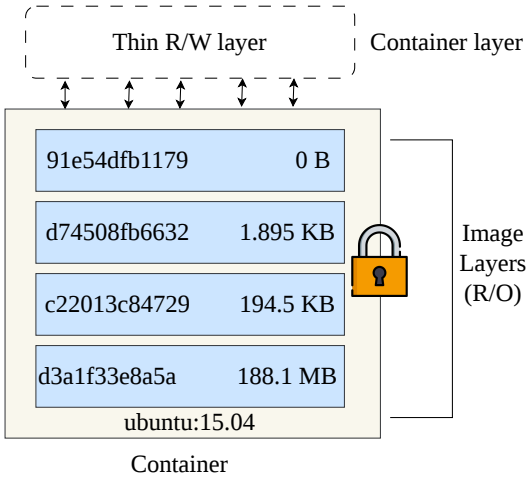
Sau khi tải về, mỗi layer được giải nén vào một thư mục riêng trong vùng lưu trữ của Docker. Khi tạo container, các thư mục này được hợp nhất thành một cây thư mục duy nhất nhờ hệ thống tệp hợp nhất; nếu cùng một tệp xuất hiện ở nhiều layer, phiên bản ở layer trên cùng sẽ được nhìn thấy. Vì mỗi layer được định danh theo nội dung, hai layer giống hệt nhau luôn có cùng mã định danh và chỉ được lưu một lần trên đĩa. Nhờ đó, các image xây dựng trên cùng một base image, chẳng hạn cùng dùng `python:3.11-slim`, sẽ dùng chung các layer nền và mỗi image chỉ bổ sung phần dữ liệu riêng của ứng dụng.

IV.2. Docker Container

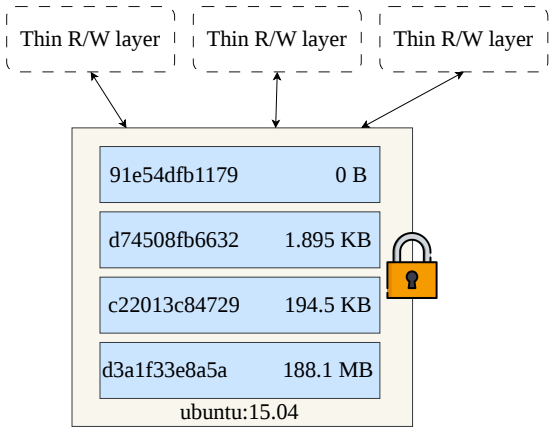


Hình 9: Các container chạy trên Docker và dùng chung kernel của máy chủ.

Container là một instance đang chạy của image. Mỗi container đóng gói ứng dụng cùng các thư viện phụ thuộc và hoạt động như một tiến trình độc lập trong user space của hệ điều hành máy chủ. Như minh họa trong Hình 9, nhiều container có thể cùng chạy trên một máy và dùng chung kernel của hệ điều hành máy chủ, thay vì mỗi ứng dụng cần một hệ điều hành khách riêng như máy ảo. Nhờ vậy, image của container thường chỉ có kích thước vài chục MB, container khởi động trong vài giây, và một máy chủ có thể vận hành nhiều ứng dụng hơn với ít tài nguyên hơn.



Hình 10: Container layer nằm trên cùng, các layer của image ở chế độ read-only.



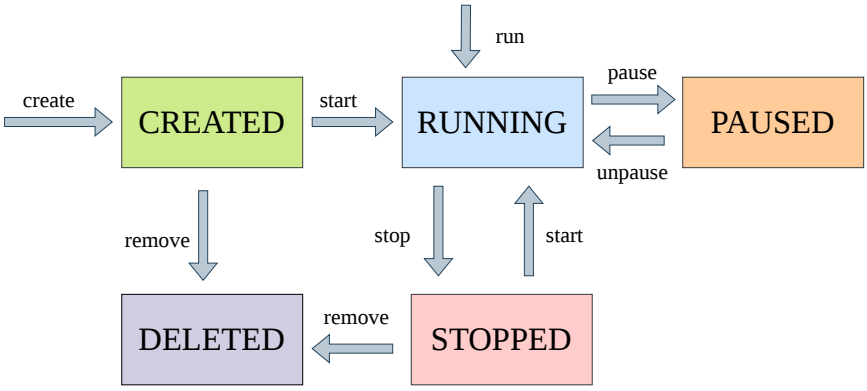
Hình 11: Nhiều container tạo từ một image dùng chung các layer read-only.

Container layer và cơ chế copy-on-write. Khi tạo container từ một image, Docker không sao chép các layer của image mà chỉ thêm một writable layer (còn gọi là container layer) lên trên cùng, như thể hiện trong Hình 10. Mọi thao tác tạo, sửa hoặc xóa tệp bên trong container đều chỉ tác động lên layer này. Khi tiến trình cần sửa một tệp đã có sẵn trong image, tệp đó được sao chép lên writable layer trước khi chỉnh sửa, cơ chế này gọi là copy-on-write. Các layer của image vì thế không bao giờ bị thay đổi và có thể được dùng chung cho nhiều container cùng lúc.

Do mỗi container có writable layer riêng, nhiều container được tạo từ cùng một image có thể dùng chung toàn bộ phần read-only (Hình 11) nhưng vẫn giữ trạng thái dữ liệu riêng biệt. Lệnh `docker`

`ps -s` thể hiện rõ điều này qua hai giá trị: `size` là dung lượng writable layer của từng container, còn `virtual` là tổng của writable layer và các layer image bên dưới. Vì các container dùng chung phần image, không thể cộng các giá trị `virtual` để tính tổng dung lượng đĩa. Writable layer bị xóa cùng với container, vì vậy container được xem là thực thể tạm thời, và dữ liệu cần lưu lâu dài phải được đặt trong volume.

Vòng đời của container. Hình 12 mô tả các trạng thái chính của một container và lệnh dùng để chuyển giữa chúng:



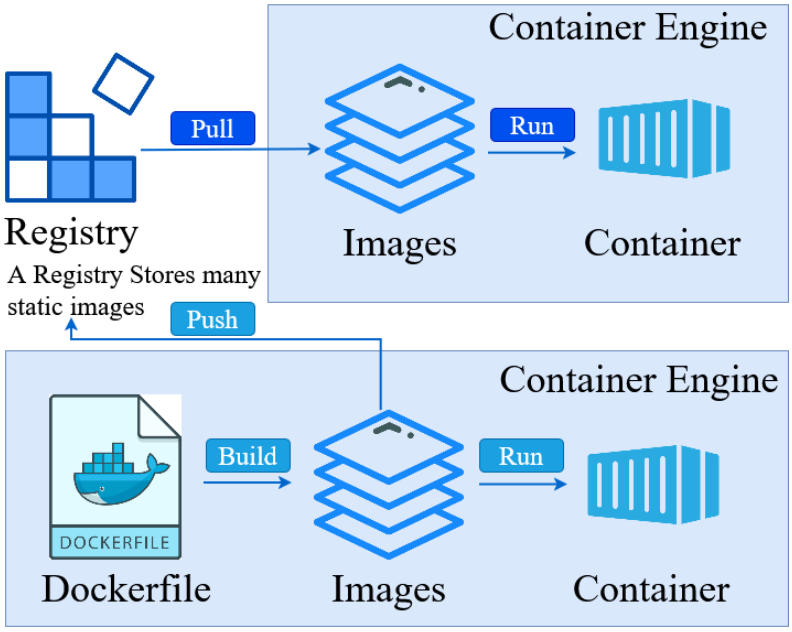
Hình 12: Tổng quan về vòng đời của container.

- **Created**: lệnh `docker create` tạo container cùng writable layer và cấu hình, nhưng tiến trình chính chưa chạy.
- **Running**: lệnh `docker start` khởi chạy tiến trình chính. Lệnh `docker run` gộp hai bước tạo và khởi chạy, đồng thời tự tải image về nếu máy chưa có, nên đưa container thẳng vào trạng thái này.
- **Paused**: lệnh `docker pause` tạm dừng toàn bộ tiến trình trong container nhưng vẫn giữ nguyên bộ nhớ; `docker unpause` cho container chạy tiếp từ đúng điểm đã dừng.
- **Stopped**: lệnh `docker stop` gửi tín hiệu `SIGTERM` đến tiến trình chính để yêu cầu dừng, và nếu tiến trình không kết thúc sau một khoảng thời gian chờ nhất định, Docker sẽ gửi tiếp `SIGKILL` để buộc dừng. Container cũng rơi vào trạng thái này khi tiến trình chính tự kết thúc. Trong kết quả của `docker ps -a`, trạng thái này hiển thị là `Exited` kèm mã thoát. Container đã dừng vẫn giữ writable layer và có thể khởi động lại bằng `docker start`.
- **Deleted**: lệnh `docker rm` xóa container cùng writable layer của nó. Image gốc không bị ảnh hưởng.

Vòng đời của container gắn liền với tiến trình chính bên trong cụ thể container chỉ chạy chừng nào tiến trình này còn chạy. Vì vậy, ứng dụng nên xử lý tín hiệu `SIGTERM` để đóng kết nối và ghi dữ liệu an toàn trước khi thoát.

IV.3. Docker Registry

Docker Registry là hệ thống lưu trữ và phân phối Docker Image. Thay vì chỉ tồn tại trên máy tính cá nhân, image được đẩy lên registry và tải về từ máy chủ, hệ thống CI/CD hoặc máy của các thành viên khác khi cần. Giao thức giao tiếp với registry được chuẩn hóa trong OCI Distribution Specification, nhờ đó Docker có thể làm việc với bất kỳ registry nào tuân thủ chuẩn này.



Hình 13: Vai trò của registry trong quy trình làm việc với Docker: image được build và push lên registry, sau đó được pull về để chạy trên các môi trường khác.

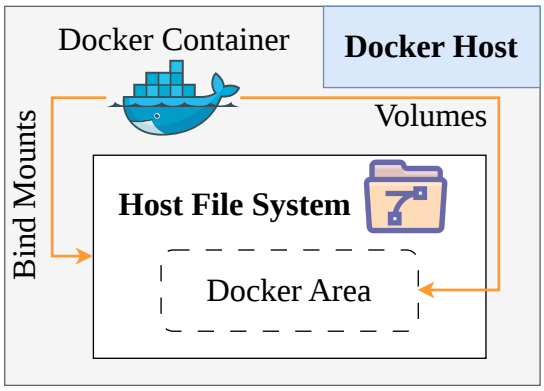
Registry được tổ chức theo ba cấp. Cấp cao nhất là registry, xác định bởi tên miền như `docker.io` hay `ghcr.io`. Mỗi registry chứa nhiều repository, mỗi repository chứa các phiên bản khác nhau của cùng một image, phân biệt bằng tag hoặc digest. Ví dụ, repository `python` trên Docker Hub chứa các tag như `3.11`, `3.11-slim`, `3.12-alpine`.

Docker Hub là registry mặc định và lớn nhất, chứa các image chính thức do Docker và cộng đồng duy trì. Ngoài ra còn có các registry công khai khác như GitHub Container Registry (GHCR). Với image nội bộ, doanh nghiệp thường dùng registry riêng để kiểm soát quyền truy cập và quét lỗ hổng bảo mật. Hình 14 minh họa một số dịch vụ container registry phổ biến hiện nay, bao gồm các nền tảng đám mây như Amazon ECR, Google Artifact Registry, Azure Container Registry, hoặc các giải pháp mã nguồn mở tự vận hành như Harbor.



Hình 14: Một số nền tảng Container Registry phổ biến được sử dụng để lưu trữ và phân phối image.

IV.4. Docker Volume & Bind Mount



Hình 15: So sánh volume và bind mount. Volume nằm trong vùng lưu trữ do Docker quản lý, còn bind mount trỏ đến một đường dẫn bất kỳ trên hệ thống tệp của máy chủ.

Một vấn đề xảy ra với writable layer đó là: mỗi container có một writable layer riêng, đây là nơi mọi thay đổi (tạo file, ghi database, cập nhật config) được lưu. Tuy nhiên, layer này gắn chặt với vòng đời của container. Khi người dùng chạy `docker rm` để xoá container, writable layer cũng bị xóa theo. Tất cả dữ liệu bên trong đều biến mất. Và khi người dùng tạo một container mới từ cùng một image trước đó, Docker sẽ cung cấp một container hoàn toàn mới, chứ không phải hệ thống tệp nội bộ của container cũ. Vì vậy đối với dữ liệu cần được lưu giữ lâu dài, Docker cung cấp cho chúng ta hai tùy chọn chính là Docker volume và Bind mount. Hình 15 minh họa hai kiểu gắn kết phổ biến nhất là volume và bind mount.

Docker Volume

Volume là kiểu lưu trữ được Docker khuyến nghị cho dữ liệu cần lưu lâu dài. Volume được tạo và quản lý hoàn toàn bởi Docker, nằm trong vùng lưu trữ riêng của Docker trên máy chủ (thường là `/var/lib/docker/volumes` trên Linux), và có vòng đời độc lập với container: xóa container không làm mất volume. Một volume có thể được gắn đồng thời vào nhiều container. Khi gắn một volume rỗng vào thư mục đã có dữ liệu trong image, Docker sẽ sao chép dữ liệu đó vào volume, giúp khởi tạo dữ liệu ban đầu thuận tiện.

Ví dụ sử dụng volume cho PostgreSQL

```
docker volume create pgdata                # create a named volume
docker volume ls                           # list volumes
docker volume inspect pgdata               # show mountpoint on the host
```

```
docker run -d --name db \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16

docker rm -f db                              # the container is gone...
docker run -d --name db2 \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16                                # ...but the data is still in pgdata

docker volume rm pgdata                      # only works when no container uses it
```

Tham số `-v pgdata:/var/lib/postgresql/data` gắn volume vào thư mục dữ liệu của PostgreSQL, nên dữ liệu được ghi vào volume thay vì writable layer. Khi container `db` bị xóa, volume vẫn còn, và container `db2` gắn lại volume đó sẽ thấy toàn bộ dữ liệu cũ. Lệnh `docker volume rm` chỉ thực hiện được khi không còn container nào dùng volume.

Bind mount

Bind mount gắn trực tiếp một tệp hoặc thư mục bất kỳ trên máy chủ vào container. Kiểu gắn kết này phụ thuộc vào cấu trúc thư mục của máy chủ nên kém linh hoạt khi chuyển giữa các môi trường, nhưng rất hữu ích khi phát triển: mã nguồn được sửa trên máy và container thấy thay đổi ngay mà không cần build lại image. Vì container có quyền ghi vào thư mục được gắn, nên thêm tùy chọn `readonly` khi container chỉ cần đọc.

Ví dụ sử dụng bind mount

```
# mount the local source code into the container (changes are visible immediately)
docker run -d --name dev -p 8000:8000 \
  --mount type=bind,source="$(pwd)"/src,target=/app/src \
  my-app:1.0

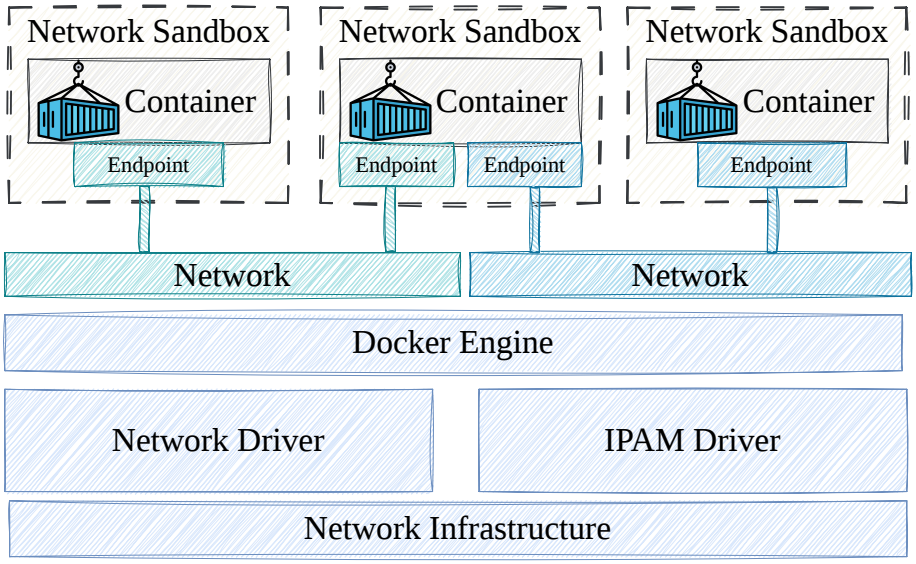
# short syntax with read-only access for a config file
docker run -d --name web \
  -v "$(pwd)"/nginx.conf:/etc/nginx/nginx.conf:ro \
  nginx
```

Lệnh đầu gắn thư mục `src` trên máy vào `/app/src` trong container, nên mọi thay đổi mã nguồn trên máy được container thấy ngay. Lệnh thứ hai gắn một tệp cấu hình đơn lẻ, hậu tố `:ro` chỉ cho container quyền đọc tệp này.

IV.5. Docker Network

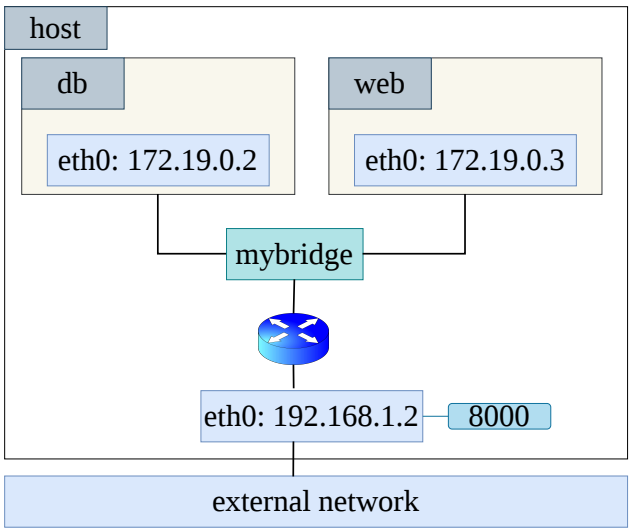
Mỗi container có một network namespace riêng, nghĩa là có giao diện mạng, địa chỉ IP, bảng định tuyến và cấu hình DNS riêng, tách biệt với máy chủ và với các container khác. Docker Network cung cấp cơ chế kết nối các container với nhau và với mạng bên ngoài. Hệ thống mạng của Docker được thiết kế theo mô hình Container Network Model (CNM), mô hình này chuẩn

hóa các bước cần thiết nhằm cung cấp khả năng kết nối mạng cho container thông qua việc sử dụng nhiều network driver khác nhau, như minh họa trong hình 16.



Hình 16: Container Network Model (CNM). Mỗi container có một network sandbox, kết nối vào network thông qua endpoint; Docker Engine dùng network driver để tạo network và IPAM driver để cấp phát địa chỉ IP.

IV.5.1. Các loại network trong Docker



Hình 17: Mô hình Bridge network kết nối các container thông qua mạng nội bộ ảo.

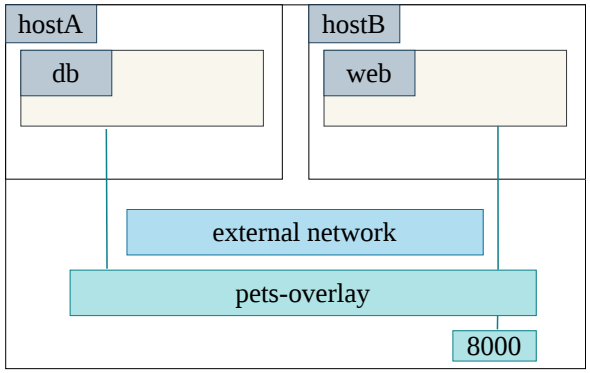
Bridge network

Đây là loại mạng sử dụng một software bridge để cho phép các container cùng kết nối vào mạng đó có thể giao tiếp với nhau. Bridge network được sử dụng cho các container đang chạy trên cùng một máy chủ Docker daemon. Các container kết nối với bridge network có thể giao tiếp với nhau thông qua địa chỉ IP hoặc tên container, nhờ vào sự hỗ trợ của máy chủ DNS tích hợp sẵn trong Docker. Ngoài ra, mạng bridge còn cho phép các container truy cập Internet thông qua các quy tắc NAT. Đây là giải pháp thường được sử dụng trong môi trường phát triển và thử nghiệm để phục vụ nhu cầu giao tiếp cục bộ giữa các container.

Docker Commands

```
docker network create -d bridge mybridge
docker run -d --net mybridge --name db redis
docker run -d --net mybridge -e DB=db -p 8000:5000 --name web chrch/web
```

Theo Hình 17 và đoạn mã trên, ứng dụng hiện đang được chạy trên máy chủ tại cổng 8000. Mạng Docker bridge cho phép container web giao tiếp trực tiếp với db thông qua tên container. Nhờ việc nằm chung một mạng, Bridge driver sẽ tự động thực hiện quá trình định vị dịch vụ (service discovery). Bên cạnh đó, toàn bộ các thao tác ánh xạ cổng, thiết lập quy tắc bảo mật và điều hướng kết nối giữa các Linux bridge đều được networking driver tự động xử lý mỗi khi các container được lập lịch hoặc tái lập lịch trên toàn bộ cụm.



Hình 18: Overlay network `pets-overlay` kết nối container `db` trên `hostA` với container `web` trên `hostB`.

Overlay network

Bridge chỉ hoạt động trong phạm vi một máy chủ. Khi ứng dụng được triển khai trên nhiều máy, mạng Overlay được sử dụng để một container trên node A có thể giao tiếp với một container trên node B (minh họa qua hình 18); mạng Overlay giúp thiết lập kết nối giữa các container. Overlay sử dụng công nghệ VXLAN, giúp tách biệt mạng container khỏi mạng vật lý bên dưới. Ưu điểm của giải pháp này là khả năng di chuyển linh hoạt tối đa giữa các môi trường mạng đám mây và on-premises khác nhau.

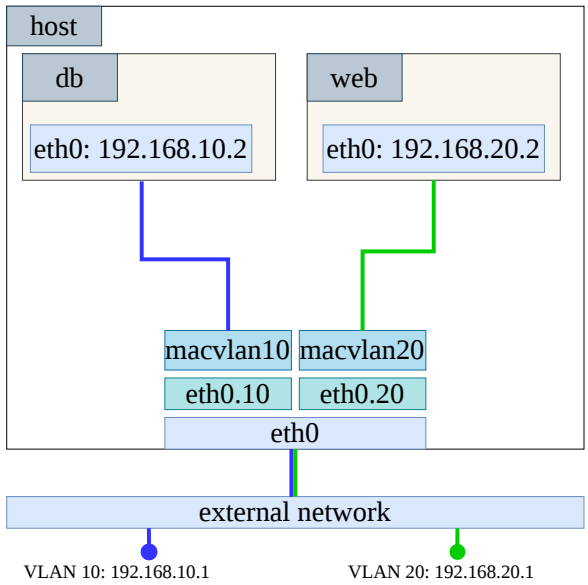
Docker Commands

```
docker network create -d overlay --opt encrypted pets-overlay
docker service create --network pets-overlay --name db redis
docker service create --network pets-overlay -p 8000:5000 \
  -e DB=db --name web chrch/web
```

Theo Hình 18 và đoạn mã trên, chúng ta vẫn đang phục vụ ứng dụng web của mình trên cổng 8000, nhưng giờ đây chúng ta đã triển khai ứng dụng phân tán trên nhiều máy chủ khác nhau. Nếu chúng ta muốn mở rộng quy mô các container web của mình, hệ thống mạng của Docker Swarm & UCP (Universal Control Plane) sẽ tự động cân bằng tải lưu lượng truy cập giúp chúng ta.

Macvlan network

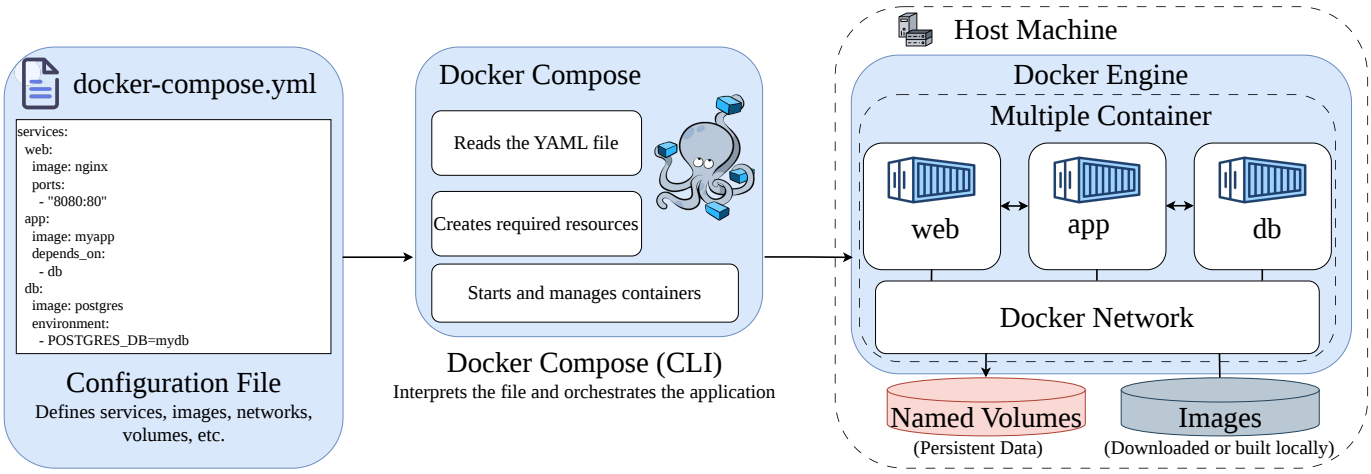
Đây là loại mạng được sử dụng để kết nối trực tiếp các ứng dụng với mạng vật lý. Bằng cách sử dụng trình điều khiển mạng Macvlan để gán địa chỉ MAC cho từng container, hệ thống cho phép container sở hữu đầy đủ TCP/IP stack. Khi đó, Docker daemon sẽ định tuyến lưu lượng truy cập đến các container dựa trên địa chỉ MAC của chúng. Người dùng có thể cô lập các Macvlan network bằng cách sử dụng các network interface vật lý khác nhau. Giải pháp này thường được áp dụng cho các ứng dụng cũ có yêu cầu về địa chỉ MAC.
Ở hình 19, các container db và web trong ví dụ này được kết nối với các mạng Macvlan khác nhau. Mỗi container nằm trên một mạng bên ngoài tương ứng và được cấp một IP ngoại vi từ mạng đó. Với thiết kế này, người quản trị có thể kiểm soát các chính sách mạng từ bên ngoài máy chủ và phân đoạn các container ở tầng L2. Các container này cũng hoàn toàn có thể được đặt chung vào một VLAN bằng cách cấu hình chúng trên cùng một mạng Macvlan.



Hình 19: Macvlan network kết hợp với 802.1Q VLAN trunking để phân tách lưu lượng container.

IV.6. Docker Compose

Các ứng dụng thực tế hiếm khi chỉ gồm một container. Một hệ thống web đơn giản đã cần một máy chủ web, một dịch vụ ứng dụng và một cơ sở dữ liệu. Nếu quản lý bằng các lệnh `docker run` riêng lẻ như ở các mục trước, người dùng phải tự tạo mạng, tạo volume, ghi nhớ thứ tự khởi động và hàng loạt tham số cho từng container, rất dễ sai sót và khó tái tạo trên máy khác. Docker Compose giải quyết vấn đề này bằng cách cho phép mô tả toàn bộ hệ thống gồm nhiều container trong một tệp YAML duy nhất, rồi khởi chạy tất cả bằng một lệnh.



Hình 20: Cơ chế hoạt động của Docker Compose. Compose đọc tệp cấu hình, tạo các tài nguyên cần thiết như network và volume, sau đó khởi chạy và quản lý các container thông qua Docker Engine.

Hình 20 mô tả quy trình làm việc của Compose gồm ba thành phần. Tệp cấu hình (thường đặt tên `compose.yaml`; tên cũ `docker-compose.yml` vẫn được hỗ trợ) khai báo các service, image, network và volume của ứng dụng. Công cụ dòng lệnh `docker compose` đọc tệp này, tạo các tài nguyên còn thiếu, sau đó yêu cầu Docker Engine khởi chạy các container theo đúng cấu hình. Mặc định, Compose tạo một mạng bridge riêng cho toàn bộ ứng dụng, nên các service gọi nhau được bằng tên.

Cấu hình Docker Compose chủ yếu xoay quanh ba thành phần:

- `services`: Định nghĩa các dịch vụ cấu thành nên ứng dụng (ví dụ: frontend, backend, database). Mỗi service đại diện cho một nhóm container chạy từ cùng một image. Tại đây, người dùng có thể khai báo chi tiết cho các container như image sử dụng hoặc cách build, cổng kết nối, các biến môi trường để linh hoạt tinh chỉnh cấu hình, cũng như các service mà nó phụ thuộc.
- `volumes`: Khai báo các phân vùng lưu trữ độc lập với vòng đời của container. Thành phần này giúp dữ liệu của ứng dụng được lưu giữ bền vững, hỗ trợ đắc lực cho việc sao lưu, quản lý và phục hồi dữ liệu.
- `networks`: Thiết lập các mạng ảo cho phép các container (trong cùng một service hoặc giữa các service khác nhau) giao tiếp và truy cập lẫn nhau thông qua tên miền hoặc địa chỉ IP. Nếu bỏ trống phần này, Compose sẽ tự động tạo và sử dụng một mạng mặc định cho dự án.

Thông qua giao diện dòng lệnh (CLI), người dùng có thể thực thi nhiều thao tác khác nhau để quản lý toàn bộ vòng đời của ứng dụng. Bảng 2 tóm tắt các lệnh `docker compose` cốt lõi và chức năng tương ứng của chúng.

Bảng 2: Các lệnh thường dùng trong Docker Compose.

Lệnh	Chức năng
`docker compose up`	Đọc cấu hình từ tệp Compose để thiết lập, tạo mới và đưa các dịch vụ vào trạng thái hoạt động.
`docker compose down`	Chấm dứt hoạt động và gỡ bỏ hoàn toàn các container cũng như network đã được Compose thiết lập.
`docker compose build`	Tiến hành build hoặc cập nhật lại image dựa trên Dockerfile của các dịch vụ.
`docker compose pull`	Tải về các image cần thiết cho dự án từ các hệ thống lưu trữ (registry) bên ngoài.
`docker compose ps`	Hiển thị danh sách toàn bộ các container trong dự án kèm theo tình trạng hoạt động hiện tại của chúng.
`docker compose logs`	Truy xuất và theo dõi nhật ký sinh ra từ các container đang vận hành.
`docker compose exec`	Chạy trực tiếp một dòng lệnh hoặc mở shell tương tác bên trong một container đang hoạt động.
`docker compose restart`	Thực hiện chu trình tắt và bật lại các container thuộc dự án.
`docker compose stop`	Tạm ngưng hoạt động của các dịch vụ nhưng vẫn bảo toàn trạng thái của container thay vì xóa bỏ.
`docker compose start`	Kích hoạt lại các container đang ở trạng thái tạm dừng (stopped) trước đó.
`docker compose config`	Kiểm tra tính hợp lệ của tệp Compose và xuất ra nội dung cấu hình dưới định dạng chuẩn hóa để dễ dàng rà soát.
`docker compose run`	Khởi tạo một container tạm thời để chạy một tác vụ đơn lẻ nhằm phục vụ kiểm tra hoặc cấu hình.
`docker compose rm`	Dọn dẹp và xóa vĩnh viễn các container đã bị dừng trước đó ra khỏi hệ thống.
`docker compose images`	Cung cấp danh sách chi tiết về các image mà dự án Compose hiện đang sử dụng.
`docker compose stats`	Giám sát trực tiếp các thông số tiêu thụ tài nguyên (như CPU, RAM...) của các container trong thời gian thực.

Để hiểu rõ hơn cách kết hợp các thành phần trên trong thực tế, chúng ta hãy cùng xem xét một ví dụ minh họa về tệp `compose.yaml` dưới đây.

Tệp compose.yaml

```yaml
services:
  web:
    image: nginx
    ports:
      - "8080:80"
    depends_on:
```

```
      - app

  app:
    build: .                        # build the image from the Dockerfile in this folder
    environment:
      DATABASE_URL: postgresql://postgres:secret@db:5432/mydb
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: mydb
      POSTGRES_PASSWORD: secret
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      retries: 5

volumes:
  pgdata:
```

Trong ví dụ trên, chỉ service `web` công bố cổng ra ngoài; `app` và `db` chỉ truy cập được từ bên trong mạng của dự án. Dữ liệu của PostgreSQL được lưu trong named volume `pgdata`, nên vẫn còn khi các container bị xóa và tạo lại. Khai báo `depends_on` với điều kiện `service_healthy` yêu cầu Compose chỉ khởi động `app` khi `db` đã vượt qua kiểm tra `healthcheck`. Nếu chỉ dùng `depends_on` dạng đơn giản như ở service `web`, Compose chỉ bảo đảm container phụ thuộc đã được khởi chạy, không bảo đảm dịch vụ bên trong đã sẵn sàng nhận kết nối. Lưu ý rằng mật khẩu được ghi trực tiếp trong tệp chỉ nhằm mục đích minh họa; trong thực tế nên đặt trong tệp `.env` và không đưa tệp này lên Git.

V. Thực hành

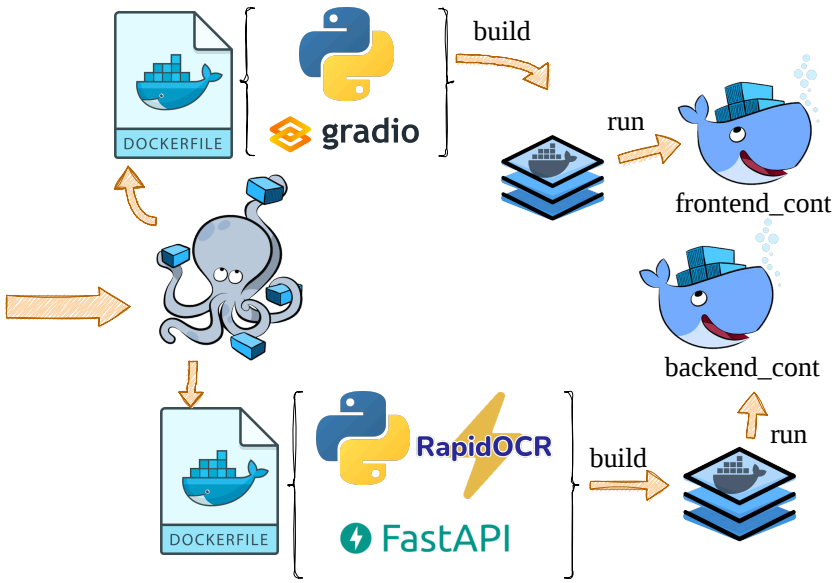
Trong phần này, chúng ta đóng gói một ứng dụng OCR (Optical Character Recognition) gồm hai service bằng Docker. Người dùng tải ảnh lên giao diện web, ứng dụng trả về các dòng chữ đọc được và vẽ khung bao quanh từng dòng trên ảnh. Trọng tâm của bài thực hành là cách viết `Dockerfile` cho từng service và cách dùng `docker-compose.yml` để build và chạy cả hệ thống bằng một lệnh.

docker-compose.yml

```
services:
  backend:
    build:
      context: ./backend
    container_name: backend_cont
    volumes:
      - ./backend/app/logs/images:/src/app/logs/images
    networks:
      - aio_mlops_net
    ports:
      - 8000:8000
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request;
urllib.request.urlopen('http://localhost:8000/health')"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s

  frontend:
    build:
      context: ./frontend
    container_name: frontend_cont
    env_file:
      - ./frontend/.env
    networks:
      - aio_mlops_net
    ports:
      - 3000:3000
    depends_on:
      backend:
        condition: service_healthy

networks:
  aio_mlops_net:
    driver: bridge
```



Hình 21: Kiến trúc Docker của ứng dụng OCR: Docker Compose đọc file `docker-compose.yml`, build image cho từng service từ `Dockerfile` tương ứng, rồi chạy hai container `frontend_cont` và `backend_cont`.

V.1. Kiến trúc tổng quan

V.1.1. Vai trò từng thành phần

Về mặt kiến trúc ứng dụng, mã nguồn của cả hai dịch vụ được thiết kế với các chức năng như sau:

- **Backend** dùng **FastAPI** để cung cấp hai endpoint, `GET /health` và `POST /ocr/predict`. Khi nhận ảnh, backend gọi **RapidOCR** để đọc chữ. RapidOCR chạy các model của PaddleOCR trên ONNX Runtime và chỉ cần CPU, nên image không cần PyTorch hay GPU.
- **Frontend** dùng **Gradio** để dựng giao diện web, gồm ô tải ảnh, nút *Run OCR*, nút *Refresh* và thanh trạng thái. Frontend gửi ảnh sang backend, sau đó vẽ khung bao lên ảnh kết quả.

V.1.2. Dockerfile của backend

Dockerfile

```
FROM python:3.11.9-slim

WORKDIR /src

COPY ./requirements.txt /src/requirements.txt

RUN pip install --no-cache-dir --upgrade -r /src/requirements.txt

# Bake the OCR models into the image so the first request doesn't download them
RUN rapidocr download_models

COPY ./app /src/app

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Ý nghĩa từng lệnh:

- `FROM python:3.11.9-slim`: chọn base image là bản Linux rút gọn đã cài sẵn Python 3.11.9. Bản `slim` bỏ bớt các công cụ không cần, nên image nhỏ hơn bản đầy đủ.
- `WORKDIR /src`: đặt thư mục làm việc trong container. Các lệnh phía sau đều chạy trong `/src`.
- `COPY ./requirements.txt ...` rồi `RUN pip install ...`: chép riêng danh sách thư viện vào trước rồi cài đặt. Tuỳ chọn `-no-cache-dir` không giữ lại cache của pip, giúp image gọn hơn.
- `RUN rapidocr download_models`: tải các file model OCR (khoảng 31 MB) ngay khi build. Model nằm sẵn trong image, nên container chạy được mà không cần internet, và request đầu tiên không phải chờ tải model.
- `COPY ./app /src/app`: chép code ứng dụng vào image. Lệnh này đặt sau bước cài thư viện có chủ đích (xem phần ghi chú về cache bên dưới).
- `EXPOSE 8000`: ghi chú rằng ứng dụng lắng nghe ở cổng 8000. Lệnh này chỉ mang tính tài liệu. Việc mở cổng ra máy thật do mục `ports` trong `docker-compose.yml` đảm nhận.
- `CMD [...]`: lệnh mặc định khi container khởi động, chạy server FastAPI bằng `uvicorn`. Địa chỉ `0.0.0.0` cho phép nhận kết nối từ bên ngoài container. Nếu để `127.0.0.1`, chỉ chính container mới truy cập được.

V.1.3. Dockerfile của frontend

Dockerfile

```
FROM python:3.11.9-slim

WORKDIR /app

COPY ./requirements.txt /app/requirements.txt

RUN pip install --no-cache-dir --upgrade -r /app/requirements.txt

COPY ./app /app

EXPOSE 3000

CMD ["python", "app.py"]
```

Dockerfile của frontend có cấu trúc giống backend, chỉ khác ba điểm: thư mục làm việc là `/app`, thư viện cài đặt là `gradio` và `requests`, và lệnh khởi động là `python app.py` ở cổng 3000. Frontend không có bước tải model vì không chạy mô hình nào.

V.1.4. File docker-compose.yml

docker-compose.yml

```
services:
  backend:
    build:
      context: ./backend
    container_name: backend_cont
    volumes:
      - ./backend/app/logs/images:/src/app/logs/images
    networks:
      - aio_mlops_net
    ports:
      - 8000:8000
    healthcheck:
      test: ["CMD", "python", "-c", "import urllib.request; urllib.request.urlopen('
                                          http://localhost:8000/health')"]
      interval: 10s
      timeout: 5s
      retries: 5
      start_period: 20s

  frontend:
    build:
      context: ./frontend
    container_name: frontend_cont
    env_file:
      - ./frontend/.env
    networks:
```

```
      - aio_mlops_net
    ports:
      - 3000:3000
    depends_on:
      backend:
        condition: service_healthy

networks:
  aio_mlops_net:
    driver: bridge
```

Thay vì phải thực thi thủ công hàng loạt lệnh rườm rà như `docker network create`, `docker build` hay `docker run`, việc sử dụng tệp `docker-compose.yml` giúp chúng ta tự động hóa và quản lý tập trung toàn bộ kiến trúc hệ thống. Dưới đây là bảng giải thích chi tiết cấu hình quan trọng đã được khai báo trong tệp:

Bảng 3: Ý nghĩa các thành phần trong cấu hình Docker Compose.

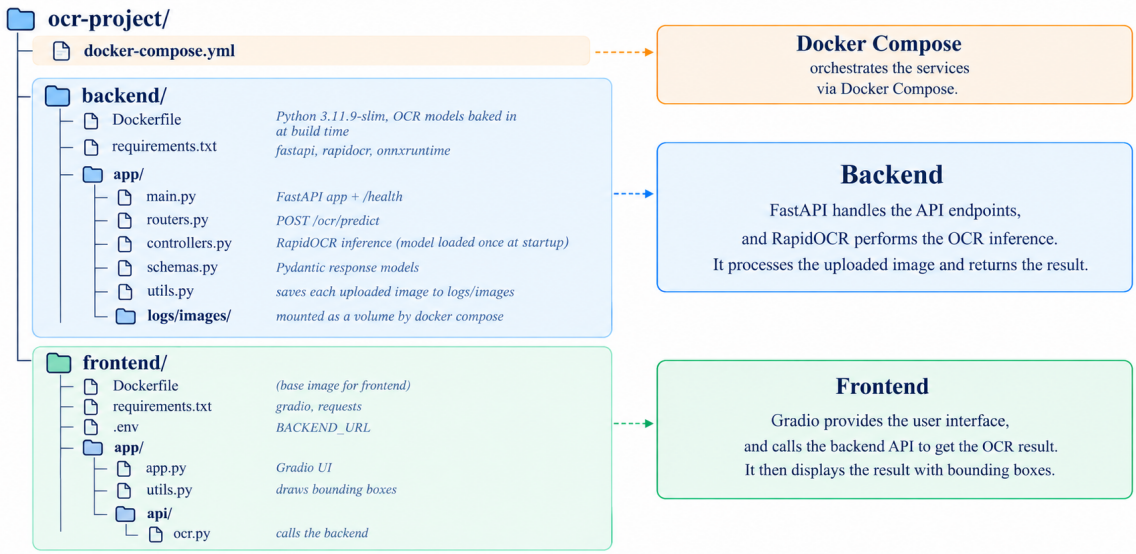
Thành phần	Ý nghĩa
`services`	Danh sách các dịch vụ (service), mỗi dịch vụ sẽ tương ứng với một container khi khởi chạy.
`build.context`	Đường dẫn đến thư mục chứa `Dockerfile` và các tệp tin cần thiết để chép vào bên trong image.
`container_name`	Chỉ định tên cố định cho container. Tên này đồng thời đóng vai trò là hostname để các container khác trong cùng mạng nội bộ (network) có thể gọi tới.
`ports`	Ánh xạ cổng kết nối theo cú pháp `cổng_máy_host:cổng_container`. Ví dụ: `3000:3000` có nghĩa là các truy cập vào `localhost:3000` trên máy thật sẽ được đẩy vào cổng 3000 của container.
`healthcheck`	Cứ 10 giây gọi GET /health của backend. Báo healthy khi gọi thành công. Image slim không có curl nên dùng python urllib.
`volumes`	Dùng để liên kết (mount) một thư mục trên máy thật vào bên trong container. Nhờ vậy, ảnh mà backend lưu vào `/src/app/logs/images` sẽ được đồng bộ ra thư mục `./backend/app/logs/images` trên máy thật và không bị mất đi ngay cả khi container bị xóa bỏ.
`env_file`	Chỉ định đường dẫn để nạp các biến môi trường từ tệp cấu hình bên ngoài. Trong trường hợp này, tệp `.env` chứa thiết lập `BACKEND_URL=http://backend_cont:8000` giúp frontend biết được chính xác địa chỉ của backend.

Tiếp tục ở trang sau

Bảng 3: Ý nghĩa các thành phần trong cấu hình Docker Compose.

Thành phần	Ý nghĩa
`networks`	Khai báo mạng nội bộ tên là `aio_mlops_net` sử dụng driver `bridge`, đồng thời cấp quyền cho cả hai service (frontend và backend) tham gia vào mạng lưới này để giao tiếp với nhau.
`depends_on`	Chỉ khởi động frontend khi backend đã healthy.

V.1.5. Cấu trúc thư mục



Hình 22: Cấu trúc thư mục của project. Mỗi service nằm trong một thư mục riêng, có `Dockerfile` và `requirements.txt` của nó. File `docker-compose.yml` ở thư mục gốc kết nối hai service lại với nhau.

Như Hình 22, project chia làm hai thư mục độc lập, `backend/` và `frontend/`. Mỗi thư mục là một build context: chỉ các file trong đó mới có thể được chép vào image của service tương ứng. Cách tổ chức này giúp mỗi service có thư viện riêng và có thể build, chạy hoặc khởi động lại riêng. Hai service chỉ giao tiếp với nhau qua HTTP. Thư mục `backend/app/logs/images/` là nơi volume được gắn vào, dùng để lưu lại các ảnh người dùng đã tải lên.

VI. Triển khai

Yêu cầu

Máy cần cài Docker Engine kèm Docker Compose, hoặc Docker Desktop trên Windows và macOS. Không cần cài Python hay GPU, vì mọi thứ đều nằm trong image.

Bước 1: Build và chạy. Tại thư mục gốc của project, chạy:

Docker Commands

```
docker compose up --build -d
```

Tuỳ chọn `-build` yêu cầu build lại image trước khi chạy. Tuỳ chọn `-d` chạy container ở chế độ nền. Lần build đầu tiên mất vài phút để tải base image, thư viện và model OCR. Các lần sau nhanh hơn nhiều nhờ layer cache.

Bước 2: Kiểm tra container.

Docker Commands

```
docker compose ps
```

Bước 3: Sử dụng ứng dụng.

- Giao diện web: mở `http://localhost:3000`, tải ảnh lên rồi bấm *Run OCR*. Bấm *Refresh* để xoá kết quả và thử ảnh khác.
- Tài liệu API: mở `http://localhost:8000/docs` để xem và gọi thử các endpoint của backend.
- Gọi API trực tiếp từ terminal:

Terminal Command

```
curl -F "file_upload=@sample.png" http://localhost:8000/ocr/predict
```

Bước 4: Xem log và dừng hệ thống.

Docker Commands

```
docker compose logs -f     # follow logs of both containers
docker compose down        # stop and remove the containers
```

Lệnh `docker compose down` xoá container và network, nhưng giữ lại image, nên lần chạy sau chỉ cần `docker compose up -d`. Các ảnh đã lưu trong volume `backend/app/logs/images` cũng không bị mất.

VII. Tài liệu tham khảo

- https://docs.docker.com/get-started/
- https://www.docker.com/resources/what-container/
- https://www.docker.com/blog/understanding-docker-networking-drivers-use-cases/
- https://docs.docker.com/build/cache/

Source code: https://github.com/ThuanNaN/packaging-ai-inference-with-docker

- Hết -